

HRCore Backend ve Supabase Teknik Raporu

1. Backend Yaklaşımımız

HRCore projesinde klasik anlamda ayrı bir Node.js, Express, Laravel veya Django backend yazılmadı. Bunun yerine backend ihtiyacı Supabase ile karşılandı. Supabase bu projede Backend-as-a-Service görevi gördü.

Yani uygulamanın backend tarafında şu ihtiyaçlar Supabase üzerinden çözüldü:

- Kullanıcı girişi
- Oturum yönetimi
- Veritabanı tabloları
- Rol bazlı veri erişimi
- Dosya depolama
- Güvenlik politikaları
- Canlı veri doğrulama

Frontend React uygulaması doğrudan Supabase JS Client üzerinden Supabase API'lerine bağlandı. Bu mimaride React sadece kullanıcı arayüzünü ve kullanıcı etkileşimini yönetir. Verinin saklanması, kimlik doğrulama ve güvenlik kuralları Supabase tarafında çalışır.

2. Supabase Nedir?

Supabase, PostgreSQL veritabanı üzerine kurulu açık kaynaklı bir backend platformudur. Supabase projeye şu temel servisleri sağlar:

Supabase bileşeni	Görevi
PostgreSQL Database	Uygulama verilerini saklar
Auth	Kullanıcı kayıt/giriş ve session yönetimi sağlar
Storage	Dosya ve doküman saklar
Row Level Security	Satır bazlı veri erişim güvenliği sağlar
JavaScript Client	Frontend'in Supabase ile konuşmasını sağlar

Supabase dokümantasyonunda platformun Postgres database, Auth, Storage ve RLS ile entegre çalıştığı belirtilir. Storage tarafı da Postgres ve RLS politikalarıyla bütünleşik çalışır.

Resmi kaynak:

- <https://supabase.com/docs/>

3. Neden Supabase Kullandık?

Bu projede Supabase tercih edilmesinin temel nedenleri şunlardır:

1. Ayrı backend API yazmadan hızlı geliştirme sağlar.
2. PostgreSQL gibi gerçek ve güçlü bir ilişkisel veritabanı kullanır.
3. Auth sistemi hazır gelir.
4. Dosya depolama için Storage hizmeti sunar.
5. Row Level Security ile frontend'den gelen istekleri güvenli hale getirir.
6. React ile kolay entegre olur.
7. MVP projeler için hızlı ama gerçekçi bir backend altyapısı sağlar.

Bu proje için çalışanlar, izinler, maaş kayıtları, cihazlar ve dokümanlar gibi veriler ilişkisel yapı gerektirir. Supabase'in PostgreSQL tabanlı olması bu yüzden uygundur.

4. Supabase Projede Nerelerde Kullanıldı?

Projede Supabase şu dosyalarda doğrudan veya dolaylı kullanıldı:

```
src/lib/supabaseClient.ts

src/features/auth/AuthProvider.tsx
src/features/auth/profileService.ts
src/features/auth/ProtectedRoute.tsx

src/features/employees/employeeService.ts
src/features/leave/leaveRequestService.ts
src/features/salary/salaryService.ts
src/features/devices/deviceService.ts
src/features/documents/documentService.ts
```

React sayfaları doğrudan Supabase sorgusu yazmak yerine bu servis dosyalarını çağırır. Böylece veri erişimi UI kodundan ayrılmış olur.

5. Supabase Client Yapısı

Supabase bağlantısı şu dosyada kuruldu:

```
src/lib/supabaseClient.ts
```

Mantık şu şekildedir:

```
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY

export const supabase = isSupabaseConfigured
  ? createClient(supabaseUrl, supabaseAnonKey)
  : null
```

Burada `VITE_SUPABASE_URL` ve `VITE_SUPABASE_ANON_KEY` `.env.local` dosyasından okunur.

Önemli güvenlik noktası:

- Frontend'de sadece public anon key kullanılır.
- Service role key frontend'e konmaz.
- Service role key RLS'i bypass edebileceği için sadece güvenli server ortamında kullanılmalıdır.

Supabase JS dokümantasyonunda client oluşturma mantığı `createClient(supabase_url, publishable_key)` şeklinde açıklanır. Ayrıca client session bilgisini varsayılan olarak local storage içinde saklayabilir.

Resmi kaynak:

- <https://supabase.com/docs/reference/javascript/auth-api>

6. Veritabanı Tasarımı

Projede backend'in ana veri modeli şu tablolara dayanır:

Tablo	Amaç
<code>profiles</code>	Auth kullanıcısının uygulama rolünü ve profil bilgisini tutar
<code>employees</code>	Çalışan kayıtlarını tutar
<code>leave_requests</code>	İzin taleplerini tutar
<code>salary_records</code>	Maaş hesaplama kayıtlarını tutar
<code>devices</code>	Cihaz envanterini tutar
<code>device_assignments</code>	Cihaz zimmet/atama kayıtlarını tutar
<code>documents</code>	Storage dosyalarının metadata kayıtlarını tutar

Bu tablolar React tarafında feature service dosyalarıyla temsil edilir.

Örneğin:

```
employees -> employeeService.ts  
documents -> documentService.ts  
devices -> deviceService.ts
```

7. Auth Sistemi

Supabase Auth kullanıcı girişi ve oturum yönetimi için kullanıldı.

Kullanıcı login olduğunda şu zincir oluşur:

```
email/password  
-> Supabase Auth  
-> auth.users.id  
-> active session  
-> profiles lookup  
-> role belirleme
```

React tarafında bu süreç `AuthProvider` içinde yönetildi.

Ana görevleri:

- Mevcut session'ı kontrol etmek
- Login olunca profile kaydını yüklemek
- Logout olunca session ve profile bilgisini temizlemek
- Context üzerinden tüm uygulamaya auth durumunu dağıtmak

Kullandığımız Supabase Auth fonksiyonları:

```
supabase.auth.getSession()  
supabase.auth.onAuthStateChange()  
supabase.auth.signInWithPassword()  
supabase.auth.signOut()  
supabase.auth.getUser()
```

Supabase Auth dokümantasyonu Auth'un authorization tarafında RLS ile birlikte kullanılabildiğini belirtir.

Resmi kaynak:

- <https://supabase.com/docs/guides/auth/>

8. Profile ve Employee Eşleme Mantığı

Bu projede sadece Supabase Auth kullanıcısı yeterli değildir. Çünkü Auth kullanıcısı teknik login kimliğidir. Uygulamanın rol ve çalışan bilgisi ayrıca tutulur.

Doğru kimlik zinciri:

```
auth.users.id
= profiles.id
= employees.profile_id
```

Bu zincirin anlamı:

- `auth.users.id` : Supabase'in login kullanıcısı
- `profiles.id` : Uygulama içi profil ve rol kaydı
- `employees.profile_id` : Bu profilin hangi çalışan kaydına ait olduğunu gösterir

Örneğin employee rolündeki Aslı kullanıcısında başlangıçta profile kaydı eksikti. Sistem e-posta fallback'i ile çalışanı bulabiliyordu ama ideal zincir kırık durumdaydı.

Düzeltilme sonrası:

```
profile=OK role=employee
employee_by_profile=OK
```

Bu backend açısından kritik bir düzeltmedir. Çünkü rol bazlı erişim için `profiles.role` , kişisel veri erişimi için `employees.profile_id` kullanılır.

9. Rol Bazlı Yetkilendirme

Projede üç temel rol vardır:

Rol	Backend/Veri erişimi mantığı
<code>admin_hr</code>	Tüm çalışan, izin, cihaz, doküman ve maaş verilerini yönetir
<code>manager</code>	Kendi takımındaki çalışanları ve izinleri görür
<code>employee</code>	Sadece kendi profilini, izinlerini, cihazlarını ve dokümanlarını görür

Frontend tarafında route koruması `ProtectedRoute` ile yapılır. Ancak asıl güvenlik backend tarafında Supabase RLS politikaları ile sağlanır.

Önemli ayrım:

```
Frontend route guard = kullanıcı deneyimi ve ekran erişimi
Supabase RLS = gerçek veri güvenliği
```

Yani kullanıcı tarayıcıda route'u manipüle etse bile Supabase RLS doğruysa yetkisiz satırları okuyamaz.

10. Row Level Security Nedir?

Row Level Security, PostgreSQL'in satır bazlı güvenlik sistemidir. Supabase bunu doğrudan kullanır. RLS sayesinde bir tablodaki her satır için "bu kullanıcı bu satırı görebilir mi, ekleyebilir mi, güncelleyebilir mi, silebilir mi?" sorusu cevaplanır.

Supabase dokümantasyonuna göre exposed schema içindeki tablolarda RLS etkin olmalıdır. `public` schema varsayılan olarak API'ye açık olduğu için bu özellikle önemlidir.

Resmi kaynaklar:

- <https://supabase.com/docs/guides/database/postgres/row-level-security>
- <https://supabase.com/docs/learn/auth-deep-dive/auth-row-level-security>

Temel RLS örneği:

```
create policy "Users can read own rows"
on table_name
for select
to authenticated
using (
  auth.uid() = user_id
);
```

Supabase dokümantasyonu `auth.uid()` fonksiyonunun isteği yapan kullanıcının ID'sini döndürdüğünü açıklar. Kullanıcı giriş yapmamışsa `auth.uid()` null döner. Bu yüzden policy yazarken `to authenticated` kullanmak ve gerektiğinde `auth.uid() is not null` kontrolü yapmak önemlidir.

11. Bu Projede RLS'e Nerede İhtiyaç Duyduk?

RLS'e ihtiyaç duyduğumuz yerler:

1. Employee sadece kendi çalışan kaydını görmeli.
2. Employee başka çalışanın verisini görememeli.
3. Manager sadece kendi takımındaki çalışanları görmeli.
4. Manager takım dışındaki çalışanları görememeli.
5. Admin/HR tüm verileri yönetebilmeli.
6. Dokümanlar sadece ilgili kişi/rol tarafından görülebilmeli.
7. Storage dosyaları herkes tarafından açıkça listelenmemeli.
8. Employee kendi dokümanını silebilmeli.

Canlı doğrulamada şu sonuçları aldık:

```
employee_read_self=ALLOW
employee_read_other=DENY
manager_read_team_member=ALLOW
manager_read_non_team_member=DENY
```

Bu, RLS sınırlarının temel okuma senaryolarında doğru çalıştığını gösterdi.

12. RLS'te Tespit Ettiğimiz Önemli Sorun

Doküman silme özelliği eklenince şu problem ortaya çıktı:

```
employee cleanup_document=ERR no_rows_deleted
admin_hr cleanup_document=OK
```

Bu ne anlama gelir?

Employee frontend'den silme işlemini başlatabiliyordu. Storage dosyası silinebiliyordu. Ancak `documents` tablosundaki metadata satırı silinmiyordu.

Sebebi:

```
public.documents tablosunda employee için DELETE policy eksikti.
```

Bu yüzden UI ilk başta listeden kaldırıyordu, panel değişince veri tekrar Supabase'den geldiği için doküman geri görünüyordu.

Bu problemi gidermek için frontend tarafında şu iyileştirme yapıldı:

```
Delete çağrısından sonra gerçekten satır silindi mi kontrol edilir.
Satır silinmediyse UI başarı gibi davranmaz.
```

Gerekli policy:

```
create policy "employees can delete own documents"
on public.documents
for delete
to authenticated
using (
  uploaded_by = auth.uid()
  or exists (
    select 1
    from public.employees e
    where e.id = documents.employee_id
    and e.profile_id = auth.uid()
  )
);
```

Bu policy ile employee:

- Kendi yüklediği dokümanı silebilir.
- Kendi employee kaydına bağlı dokümanı silebilir.

13. Supabase Storage Nedir?

Supabase Storage dosya depolama servsidir. Bu projede çalışan dokümanları ve cihaz faturaları gibi dosyalar için kullanıldı.

Kullandığımız bucketlar:

Bucket	Amaç
employee-documents	Çalışan dokümanları
device-invoices	Cihaz faturaları

Supabase Storage dokümantasyonu dosyaların organize edilebildiğini, servis edilebildiğini ve erişim politikaları ile yönetilebildiğini belirtir.

Resmi kaynaklar:

- <https://supabase.com/docs/guides/storage>
- <https://supabase.com/docs/guides/storage/serving/downloads>

14. Doküman Upload/Download Süreci

Doküman modülünde backend akışı şu şekildedir:

```
Kullanıcı dosya seçer
-> React dosya validasyonu yapar
-> Supabase Storage'a upload edilir
-> public.documents tablosuna metadata insert edilir
-> Liste güncellenir
```

Storage path formatı:

```
employees/{employee_id}/{document_type}/{timestamp}_{safe_file_name}
```

Örnek:

```
employees/4becc053.../employee_document/1779740259096_file.txt
```

Metadata tablosunda şu bilgiler tutulur:

```
employee_id
device_id
document_type
file_name
storage_bucket
storage_path
uploaded_by
created_at
updated_at
```

Bu ayrım önemlidir:

```
Storage = gerçek dosya
documents tablosu = dosyanın uygulama içindeki kaydı
```

15. Signed URL Mantığı

Dosyalar private bucket içinde tutulduğunda doğrudan herkese açık link verilmez. Bunun yerine geçici signed URL üretilir.

Projede kullanılan akış:

```
supabase.storage
  .from(bucket)
  .createSignedUrl(path, 60)
```

Bu, 60 saniyelik geçici indirme linki üretir.

Supabase dokümantasyonu private dosyalar için `createSignedUrl` kullanılabileceğini açıklar.

Resmi kaynak:

- <https://supabase.com/docs/guides/storage/serving/downloads>

Bu yöntem güvenlik açısından daha doğrudur. Çünkü dosya herkesin erişimine açık kalmaz.

16. Storage Smoke Test Sürecimiz

Canlı doğrulamada üç rol için storage testleri yaptık:

- Admin/HR
- Manager
- Employee

Test akışı:

```
login
-> profile eşleşmesi
-> employee eşleşmesi
-> test dosyası upload
-> documents metadata insert
-> signed URL üret
-> signed URL ile indir
-> metadata cleanup
-> storage cleanup
```

Başarılı sonuç:

```
storage_upload=OK
document_insert=OK
signed_url=OK
signed_download=OK
cleanup_storage=OK
```

Employee için metadata delete policy eksik olduğu için ilk testte `cleanup_document=ERR no_rows_deleted` gördük. Bu bize RLS DELETE policy ihtiyacını net gösterdi.

17. Metadata ve Storage Tutarlılığı

Canlı veride bir doküman kaydı incelendi:

```
file_name=Sıcaklık Sensorları.docx
employee=Aylin Demir
uploaded_by=Mehmet Admin
```

Ancak storage tarafında dosya bulunamadı:

```
storage_object=ERR Object not found
```

Bu şu anlama gelir:

```
documents tablosunda metadata var
ama bucket içinde gerçek dosya yok
```

Bu güvenlik açığı değildir; veri tutarlılığı problemidir.

Muhtemel sebepler:

1. Dosya sonradan silinmiştir.
2. Upload yarım kalmıştır.
3. Eski/manual işlem metadata bırakmıştır.
4. Dosya farklı path'e yüklenmiştir.

Bu yüzden doküman kayıtları için iki aşamalı düşünmek gerekir:

```
DB kaydı var mı?  
Storage objesi var mı?  
Signed URL üretilebiliyor mu?
```

18. CRUD Mantığı

Backend açısından uygulamadaki temel işlemler CRUD modeline uyar:

İşlem	Anlam	Projedeki örnek
Create	Yeni kayıt ekleme	employee, leave request, device, document
Read	Veri okuma	dashboard, listeler, detay ekranı
Update	Veri güncelleme	employee status, leave approval, device return
Delete	Veri silme	document delete

Supabase tarafında bunlar şu methodlarla yapılır:

```
.select()  
.insert()  
.update()  
.delete()
```

React service layer bu işlemleri soyutlar.

19. Service Layer Neden Önemli?

Eğer her React sayfasında doğrudan Supabase sorgusu yazsaydık kod dağınık olurdu. Bu yüzden feature bazlı service dosyaları kullanıldı.

Örnek:

```
DocumentsPage  
-> documentService.getDocuments()  
-> Supabase
```

Bu yaklaşımın avantajları:

- UI kodu sade kalır.
- Veri erişimi tek yerde toplanır.
- Hata yönetimi standartlaşır.
- Tip tanımları tek yerde tutulur.
- Backend değişirse sayfa kodları daha az etkilenir.

20. Backend Güvenliğinde Dikkat Ettiğimiz Noktalar

Projede backend güvenliği için şu prensipler kullanıldı:

1. Frontend'e service role key konmadı.
2. Sadece anon key kullanıldı.
3. Auth session ile kullanıcı kimliği belirlendi.
4. Profile tablosuyla uygulama rolü ayrıştırıldı.
5. Employee tablosuyla gerçek çalışan eşleşmesi kuruldu.
6. ProtectedRoute ile frontend ekran koruması yapıldı.
7. RLS ile gerçek veri erişim koruması yapıldı.
8. Storage dosyaları signed URL ile indirildi.
9. Anon erişim sınırları test edildi.
10. Delete işlemlerinde gerçekten satır silindi mi kontrol edildi.

21. Supabase Web Sitesinden Kontrol Edilmesi Gereken Yerler

Supabase Dashboard üzerinde şu bölümleri kontrol ederek backend sağlığı doğrulanabilir:

21.1 Authentication

Dashboard yolu:

Supabase Dashboard -> Authentication -> Users

Burada kontrol edilecekler:

- Admin/HR kullanıcısı var mı?
- Manager kullanıcısı var mı?
- Employee kullanıcısı var mı?
- Kullanıcı e-postaları doğru mu?
- UUID değerleri `profiles.id` ile eşleşiyor mu?

21.2 Table Editor

Dashboard yolu:

Supabase Dashboard -> Table Editor

Kontrol edilecek tablolar:

```
profiles
employees
leave_requests
salary_records
devices
device_assignments
documents
```

Özellikle kontrol edilmesi gerekenler:

```
profiles.id = auth.users.id
employees.profile_id = profiles.id
employees.manager_id doğru mu?
documents.storage_path doğru mu?
```

21.3 SQL Editor

Dashboard yolu:

```
Supabase Dashboard -> SQL Editor
```

Burada RLS policy ekleme, düzeltme ve test sorguları çalıştırılır.

Employee doküman silme için gerekli policy burada eklenmelidir.

21.4 Authentication Policies / RLS Policies

Dashboard üzerinde tablo detaylarında RLS policy listesi kontrol edilmelidir.

Kontrol:

- `profiles` için select policy var mı?
- `employees` için role-based select policy var mı?
- `documents` için select/insert/delete policy var mı?
- `devices` ve `device_assignments` policy sınırları doğru mu?

21.5 Storage

Dashboard yolu:

```
Supabase Dashboard -> Storage
```

Kontrol edilecek bucketlar:

```
employee-documents
device-invoices
```

Kontrol noktaları:

- Bucketlar private mı?
- Dosya pathleri doğru formatta mı?
- Eksik obje var mı?
- `.emptyFolderPlaceholder` dışında gerçek dosyalar var mı?
- Storage policies doğru mu?

22. Süreçte Neler Yaptık?

Backend/Supabase sürecinde yaptıklarımız:

1. Supabase client bağlantısını doğruladık.
2. `.env.local` içindeki URL ve anon key değerlerini kontrol ettik.
3. Anon erişimde hangi tablolar açık/kapalı test ettik.
4. Admin/HR login doğrulaması yaptık.
5. Manager login doğrulaması yaptık.
6. Employee login doğrulaması yaptık.
7. Profile ve employee eşleşmelerini kontrol ettik.
8. Employee kullanıcısında eksik profile bağlantısını düzelttik.
9. RLS sınır testi yaptık.
10. Employee başka çalışanı göremiyor mu kontrol ettik.
11. Manager takım dışı çalışanı göremiyor mu kontrol ettik.
12. Storage upload/download testleri yaptık.
13. Signed URL ile dosya indirmenin çalıştığını doğruladık.
14. Doküman metadata ve storage obje tutarlılığını inceledik.
15. Eksik storage objesi olan eski doküman kaydını tespit ettik.
16. Doküman silme özelliğini frontend'e ekledik.
17. Employee delete policy eksikliğini canlı testte yakaladık.
18. HRCore marka değişimini uyguladık.
19. Login sayfasını sadeleştirdik.
20. Build ve lint doğrulamalarını yaptık.

23. Backend Tarafında Kalan En Önemli İş

Şu anda backend tarafında en önemli kalan iş:

```
Employee documents DELETE policy eklenmeli.
```

Gerekli SQL:

```
create policy "employees can delete own documents"
on public.documents
for delete
to authenticated
using (
  uploaded_by = auth.uid()
or exists (
  select 1
  from public.employees e
  where e.id = documents.employee_id
  and e.profile_id = auth.uid()
)
);
```

Policy eklendikten sonra şu test tekrar çalıştırılmalıdır:

```
node scripts/supabase-storage-smoke.mjs employee
```

Beklenen sonuç:

```
cleanup_document=OK
cleanup_storage=OK
```

24. Supabase Öğrenmek İçin Temel Kavramlar

Bu projeyi anlayabilmek için Supabase tarafında şu kavramları bilmek gerekir:

Kavram	Açıklama
Project URL	Supabase projesinin API adresi
Anon Key	Frontend'in kullandığı public API key
Service Role Key	RLS bypass edebilen gizli admin key
Auth User	Supabase Authentication kullanıcısı
Session	Giriş yapan kullanıcının aktif oturumu
PostgreSQL Table	Verilerin tutulduğu tablo
Row Level Security	Satır bazlı erişim kuralı
Policy	RLS için yazılan SQL kuralı
Storage Bucket	Dosyaların saklandığı klasör benzeri alan
Storage Path	Dosyanın bucket içindeki yolu
Signed URL	Private dosya için geçici erişim linki
Metadata	Dosyanın DB tarafındaki açıklayıcı kaydı

25. Akademik Değerlendirme

Bu projede backend tarafı Supabase ile modern Backend-as-a-Service yaklaşımı kullanılarak geliştirilmiştir. Aynı bir backend sunucusu yazılmamış olsa da, backend sorumlulukları ortadan kalkmamıştır. Bu sorumluluklar Supabase servisleriyle çözülmüştür.

Akademik açıdan proje şu backend yetkinliklerini göstermektedir:

- İlişkisel veri modeli kurma
- Auth ve kullanıcı rolü ayrımı yapma
- Frontend ile backend veri akışını ayırma
- RLS ile veri güvenliği tasarlama
- Dosya depolama ve metadata ilişkisi kurma
- Canlı backend doğrulaması yapma
- Policy eksiklerini testle yakalama
- Hata durumlarını UI ve backend açısından yorumlama

26. Sonuç

HRCore projesinde backend süreci Supabase üzerine kurulmuştur. Supabase Auth kullanıcı girişini, PostgreSQL tablolar veri saklamayı, Storage dosya yönetimini ve RLS politikaları veri güvenliğini sağlamıştır.

React uygulaması Supabase JS Client ile bu backend servislerine bağlanmıştır.

Bu süreçte sadece veriyi çekmekle kalmamış, canlı Supabase ortamında rol bazlı erişim, storage upload/download, signed URL, employee-manager sınırları ve doküman silme davranışı da test edilmiştir.

Sonuç olarak proje, backend tarafında gerçek bir MVP uygulamasında beklenen temel katmanlara sahiptir:

```
Auth
Database
Storage
RLS
Role-based access
Live verification
```

Tam teslim için en kritik son adım, employee doküman silme policy'sinin Supabase SQL Editor üzerinden eklenmesi ve tekrar test edilmesidir.